

Device-Claim Prototype — Engine Consumption Evidence

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Scope:** REAL
prototype proving Helix OTA can consume the `session_orchestrator` reusable engine's claim + scheduler packages as a single-owner device-claim registry (§11.4.176 exactly-once / §11.4.119 single-resource-owner). Dev-tooling only — NOT product runtime. Engine consumed BY REFERENCE (§11.4.28(C) depth-1 carve-out); the engine submodule was NOT modified. §11.4.6: every claim below is backed by captured command output in `test_output.txt` (sibling file).

1. Engine API consumed (cited exact files/functions)

Engine module `github.com/vasic-digital/session_orchestrator` (go 1.22), `constitution/submodules/session_orchestrator/`. The prototype consumes:

claim package — the single-owner exactly-once registry

- `claim.New(cfg Config) *Registry` — `claim/claim.go:167`.
`Config{Now Clock, NewID IDGen, Liveness Liveness}` (all injected, nil-safe defaults) — `claim/claim.go:53-151`.
- `(*Registry).TryClaim(resourceID, holder string, ttl time.Duration) (Claim, Outcome, error)` — `claim/claim.go:203`.
NON-BLOCKING atomic compare-and-set under one mutex (`claim/claim.go:214-239`). Outcomes GRANTED / GRANTED_EXISTING / DENIED — `claim/claim.go:86-98`. This is the exactly-once (§11.4.176-A) + single-owner (P1 / §11.4.119) primitive: two concurrent claimants of one free resource are serialized by the lock — exactly one wins GRANTED, every other gets clean DENIED.
- `(*Registry).Release(resourceID, claimID string) error` — `claim/claim.go:246`. Claim-id must match (`ErrClaimMismatch`) so a stale holder cannot release a reaped-and-re-granted claim (§9.2).

- `(*Registry).Renew(resourceID, claimID string) (Claim, error)` — `claim/claim.go:279`. Explicit TTL heartbeat.
- `(*Registry).Reap() []Claim / IsClaimed / Snapshot / Events / WriteStatus` — `claim/claim.go:315,304,384,398,419`.
- **TTL / dead-holder reap semantics** — `staleReason claim/claim.go:364`: with a Liveness proof configured the proof is authoritative (a provably-DEAD holder is reaped ANY time; a provably-ALIVE holder is NEVER reaped even past TTL — reaping a live holder would cross-contaminate its resource, §9.2); with NO Liveness proof it is a pure-TTL lease reaped once `now >= ExpiresAt()`. Absence-of-proof never becomes “assume dead” (§11.4.6) — `claim/claim.go:20-34`.

scheduler package — non-failover placement (pure composition)

- `scheduler.Schedule(reg *alias.Registry, cr *claim.Registry, workUnits []string, cfg Config) (Result, error)` — `scheduler/scheduler.go:133`. Composes `claim.FirstOperableUnclaimed` (`claim/select.go:30` — “pool minus currently-claimed minus non-operable” in priority order) + `claim.TryClaim`. Each work-unit gets exactly one operable, unclaimed alias, exactly-once; a work-unit with no claimable operable alias is returned explicitly `Unassigned` (never dropped, never double-assigned — `scheduler/scheduler.go:161-170,196-217`). `Config.Probe` is required — a nil probe is a config error, there is NO fail-open “unprobed = healthy” path (§11.4.69) — `scheduler/scheduler.go:107-109,140-142`.
- alias health layer used indirectly: `alias.NewRegistry, (*Registry).Register, alias.ProbeResult{HTTPStatus,Body,Err}, alias.VerifyToken, alias.IsOperable` (fail-closed) — `alias/registry.go:52,59, alias/health.go:80,28,155`.

Honest boundary (§11.4.6) — the absent failover spine

The engine’s same-session FAILOVER / re-homing spine (WS-C: detect a degraded holder → atomically move its work onto another) is **UNCONFIRMED / NOT implemented** in the engine (`claim/claim.go:36-38, scheduler/scheduler.go:16-22`). The prototype therefore models claim / release / single-owner / dead-holder-reap ONLY. Automatic re-homing of a crashed worker’s in-flight device test remains conductor-driven (§11.4.147) on top of the dead-holder reap this exposes — the engine gives the death SIGNAL (reap), not the automatic re-home ACTION.

2. Engine builds + tests GREEN (unmodified)

```
cd constitution/submodules/session_orchestrator && go build ./...  
&& go test ./... (captured, test_output.txt lines 4-10):
```

```
engine BUILD_EXIT=0  
ok  github.com/vasic-digital/session_orchestrator/alias  
0.006s  
ok  github.com/vasic-digital/session_orchestrator/claim  
0.418s  
ok  github.com/vasic-digital/session_orchestrator/scheduler  
0.003s  
ok  github.com/vasic-digital/session_orchestrator/supervisor  
0.066s  
engine TEST_EXIT=0
```

The engine submodule was NOT modified (read-only per task mandate).

3. Prototype design

tools/device_claim/ — module github.com/HelixDevelopment/helix_ota/tools/device_claim (go 1.22). Consumes the engine via go.mod:

```
require github.com/vasic-digital/session_orchestrator v0.0.0  
replace github.com/vasic-digital/session_orchestrator => ../.../  
constitution/submodules/session_orchestrator
```

The engine has zero external deps (helix-deps.yaml: deps: [], stdlib only), so the local replace resolves with NO network fetch and NO go.sum for it.

devclaim.go wraps claim.Registry as a **device-claim registry** modelling the real Helix case: the exclusive resource is a physical target device addressed by its **stable serial** (§11.4.111 — never an adb devices enumeration slot, reassigned on hotplug); the holder is a test-worker id.

- New(Config{Now, Liveness}) → wraps claim.New. Liveness = the consumer's fast non-blocking dead-worker proof (kill -0 / heartbeat-file mtime).

- TryAcquire(serial, worker, ttl) (Lease, Outcome, error) → TryClaim
 - a stable-serial / non-empty-worker guard (§11.4.111).
- Release / Renew / Owner / IsHeld / Reap → the engine primitives, renamed into the Helix device domain.
- PlaceWorkers(devices, workers, reachable, ttl, now) → composes scheduler.Schedule: each device registered as an alias keyed by stable serial; a device is a candidate only if reachable(serial) (the device-testing analogue of alias health, mapped to a positive-evidence probe / fail-closed UNREACHABLE) AND unclaimed. All-or-nothing, single-owner, never-drop.

It composes, does not re-implement — exactly-once / single-owner / non-blocking / TTL-reap all come from the engine; the wrapper adds only Helix-domain naming + the stable-serial guard + the reachable → probe mapping.

4. Test results (go test -race -count=1, captured test_output.txt lines 16-41)

gofmt -l clean, go vet exit 0. All 9 tests PASS under -race; re-run -count=3 also GREEN (self-cleaning state, §11.4.98).

#	Required property	Test	Result
	exactly-once — 2 concurrent claimants of one serial ⇒		
(a)	exactly one GRANTED, one DENIED (500 iterations, maximal race)	TestExactlyOnce_ConcurrentClaimants	PASS
(a)	same-worker re-acquire is idempotent (GRANTED_EXISTING, same claim id)	TestExactlyOnce_SameWorkerIdempotent	PASS
	single-owner over time — hold → deny- other → release → re-claim by different worker; wrong-claim-		
(b)		TestSingleOwner_ReleaseThenReclaim	PASS

#	Required property	Test	Result
	id release rejected (§9.2)		
(c)	deadlock-free — 60 workers × 3 devices TryAcquire/Release storm completes < 10s, never double-owns	TestDeadlockFree_MultiDeviceStorm	PASS
(c)	deadlock-free (scheduler) — 4 workers / 2 devices all-or-nothing placement terminates, 2 assigned + 2 honest Unassigned, no device double-assigned	TestDeadlockFree_PlaceWorkersStorm	PASS
(d)	TTL reap — pure-TTL lease past horizon reclaimable by another worker; denied before expiry	TestTTLReap_ExpiredLease	PASS
(d)	dead-worker reap — provably-dead holder reclaimable immediately (before TTL); provably-alive holder NEVER reaped even past TTL	TestLivenessReap_DeadWorker	PASS
guard	stable-serial (§11.4.111) — empty serial / empty worker rejected	TestStableSerialGuard	PASS
guard	fail-closed — unreachable device NEVER assigned (§11.4.69)	TestPlaceWorkers_UnreachableDeviceExcluded	PASS

ok github.com/HelixDevelopment/helix_ota/tools/device_claim
1.031s (-race -count=1)

```
ok  github.com/HelixDevelopment/helix_ota/tools/device_claim
1.060s  (-race -count=3)
```

Systematic-debugging note (§11.4.102) — one real failure caught + fixed

First -race run: TestLivenessReap_DeadWorker FAILED — dead-worker device owner="" (want worker-takeover). Root cause (proven, not guessed): the engine reaps ANY holder whose Liveness proof returns false; the test's alive map omitted worker-takeover, so its default false made the engine correctly reap the brand-new legitimate holder on the next Snapshot. This was a TEST-FIXTURE bug (§11.4.1 — a live takeover worker must report alive), NOT an engine defect; fixed by registering worker-takeover: true. Re-run GREEN. This confirms the engine's fail-closed liveness discipline is genuinely strict.

5. Verdict

The session_orchestrator claim + scheduler packages are a build-verified, test-covered, deterministic implementation of exactly the §11.4.176 exactly-once / §11.4.119 single-owner device-claim primitive Helix OTA needs for parallel device testing. This prototype consumes them BY REFERENCE with a small dev-tooling wrapper and proves all four required properties under -race. The only gap is the UNCONFIRMED absent WS-C failover spine — conductor-driven §11.4.147 respawn stays on top of the engine's dead-holder reap until that spine lands.